

CI/CD Basics with GitHub Actions

Hands-On Lab Completion Report

Name: Nigatie Sahlie | Repository: CI-CD-Project-2 | Date: July 15, 2026

Project Overview

This report documents the completion of a four-part hands-on lab covering GitHub Actions fundamentals: creating a first workflow, automated testing with a build matrix, deploying a static site to AWS S3, and adding a manual approval gate for production deployments. All work was performed on the repository Nigatie-Sahlie/CI-CD-Project-2, driven from an Ubuntu terminal and verified through the GitHub Actions dashboard.

Lab	Objective	Status
Lab 1	First GitHub Actions workflow (checkout, print commit info, create artifact)	Complete
Lab 2	Automated Python testing across 3 Python versions + linter	Complete
Lab 3	Deploy static website to AWS S3 via GitHub Actions	Complete
Lab 4	Manual approval gate before production deployment	Complete

Lab 1: First GitHub Actions Workflow

A workflow file (first-workflow.yml) was created under .github/workflows to trigger on push, pull request, and manual dispatch. It checks out the code, prints commit and repository metadata, lists repository files, generates a timestamped text file, and uploads it as a build artifact.

```
nigie@CI_CD_Project_2
❯ $ # Create the workflow file
cat > .github/workflows/first-workflow.yml << 'EOF'
name: First Workflow

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch: # Allow manual trigger

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Print commit message
        run: |
          echo "Commit message: ${ github.event.head_commit.message }"
          echo "Commit author: ${ github.event.head_commit.author.name }"
          echo "Repository: ${ github.repository }"
          echo "Branch: ${ github.ref_name }"
          echo "Runner OS: ${ runner.os }"

      - name: List files in repository
        run: ls -la

      - name: Create a file
        run: |
          echo "Workflow ran at $(date)" > workflow-info.txt
          cat workflow-info.txt

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: workflow-info
          path: workflow-info.txt
EOF
nigie@CI_CD_Project_2
❯ $ ls -la
total 12
drwxrwxr-x 3 nigie nigie 4096 Jul 15 10:57 .
drwxrwxr-x 7 nigie nigie 4096 Jul 15 10:53 ..
drwxrwxr-x 3 nigie nigie 4096 Jul 15 10:57 .github
nigie@CI_CD_Project_2
```

Figure 1 — first-workflow.yml created via terminal heredoc.

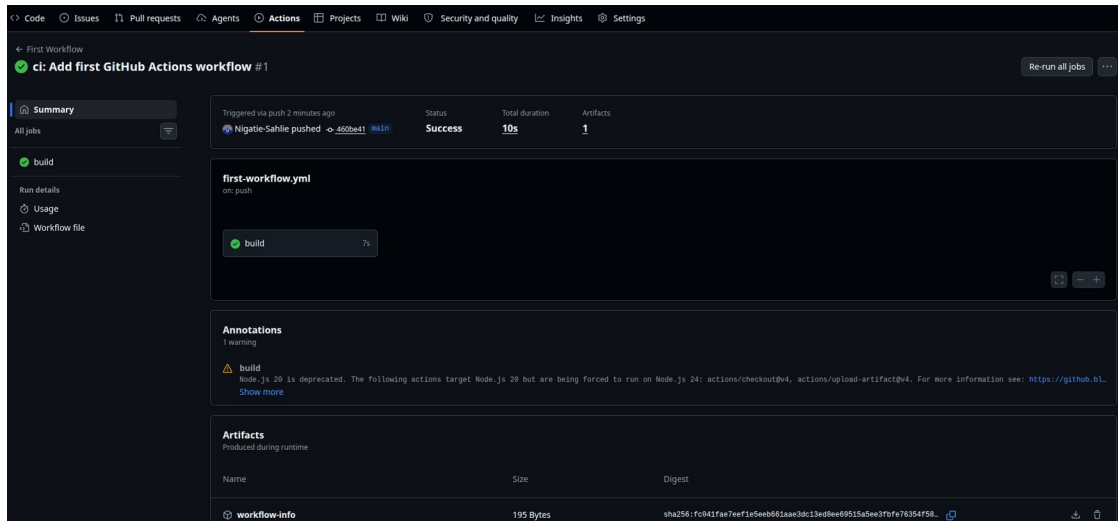


Figure 2 — Workflow run succeeded (10s); workflow-info artifact (195 Bytes) uploaded.

Lab 2: Automated Testing Workflow

A small Python application (app.py) with add, subtract, multiply, and divide functions was created alongside a pytest test suite (tests/test_app.py). The test.yml workflow runs the suite across Python 3.9, 3.10, and 3.11 using a build matrix, caches pip dependencies, and runs flake8 as a linting step. An initial run failed due to a missing conftest.py (pytest could not resolve the app module); adding an empty conftest.py at the repository root resolved it.

```

nige@CI-CD-Project-2:~$ cat > app.py << 'EOF'
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
EOF
nige@CI-CD-Project-2:~$ ls -la app.py
-rw-rw-r-- 1 nige nige 209 Jul 15 11:15 app.py
nige@CI-CD-Project-2:~$ cat > tests/test_app.py << 'EOF'
import pytest
from app import add, subtract, multiply, divide

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_subtract():
    assert subtract(5, 3) == 2
    assert subtract(0, 5) == -5

def test_multiply():
    assert multiply(2, 3) == 6
    assert multiply(-2, 3) == -6

def test_divide():
    assert divide(6, 2) == 3
    with pytest.raises(ValueError):
        divide(5, 0)
EOF
nige@CI-CD-Project-2:~$ ls -la tests/test_app.py
-rw-rw-r-- 1 nige nige 409 Jul 15 11:15 tests/test_app.py
nige@CI-CD-Project-2:~$

```

Figure 3 — app.py and tests/test_app.py created via terminal.

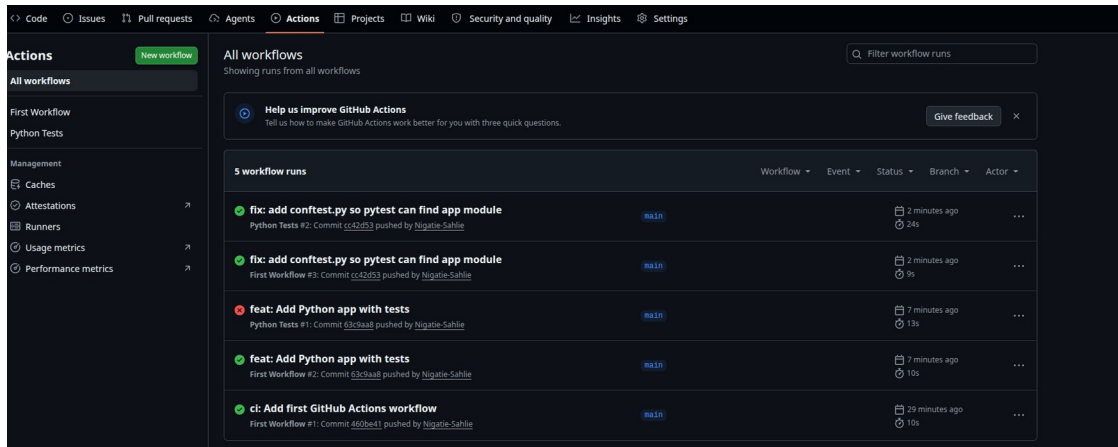


Figure 4 — Python Tests and First Workflow runs passing after the conftest.py fix.

Lab 3: Deploy to AWS S3 from GitHub Actions

An S3 bucket was created and configured for static website hosting, with Block Public Access disabled and a public-read bucket policy applied. A deploy.yml workflow was added that configures AWS credentials from repository secrets and syncs the repository's website files to the bucket on every push to index.html, *.html, *.css, or *.js. The AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY secrets were added under repository settings, and the access key was rotated after being briefly exposed during setup.

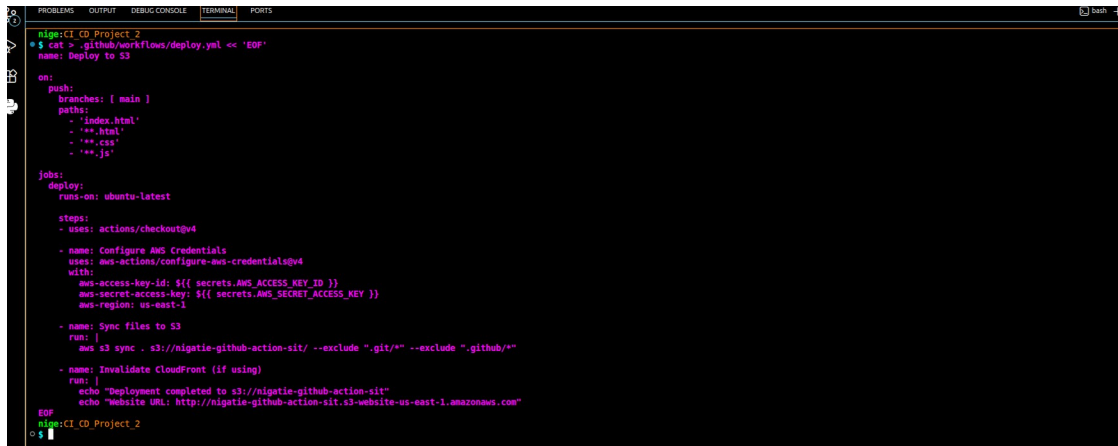


Figure 5 — deploy.yml workflow file created.

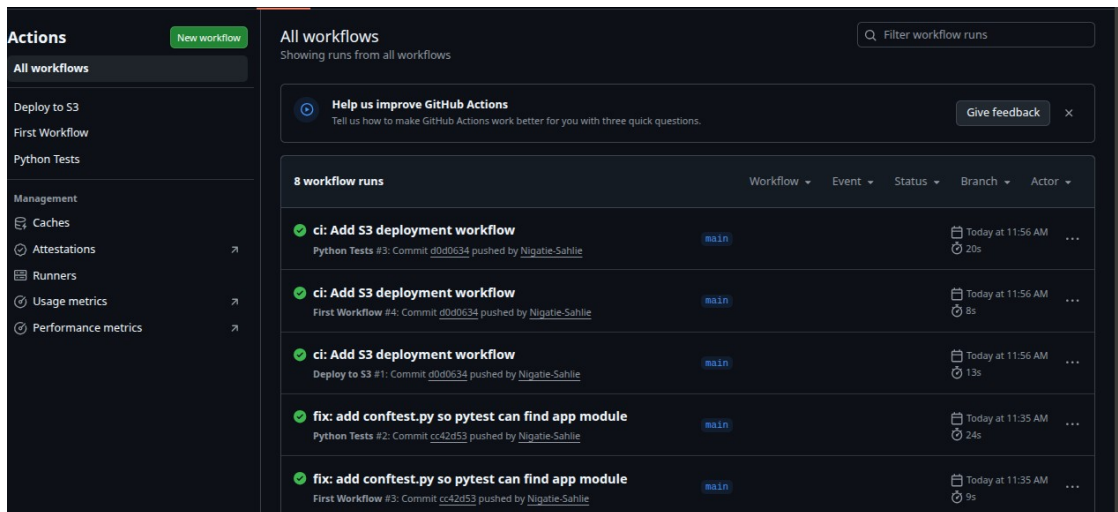


Figure 6 — Deploy to S3 job succeeded after correcting the repository secrets.

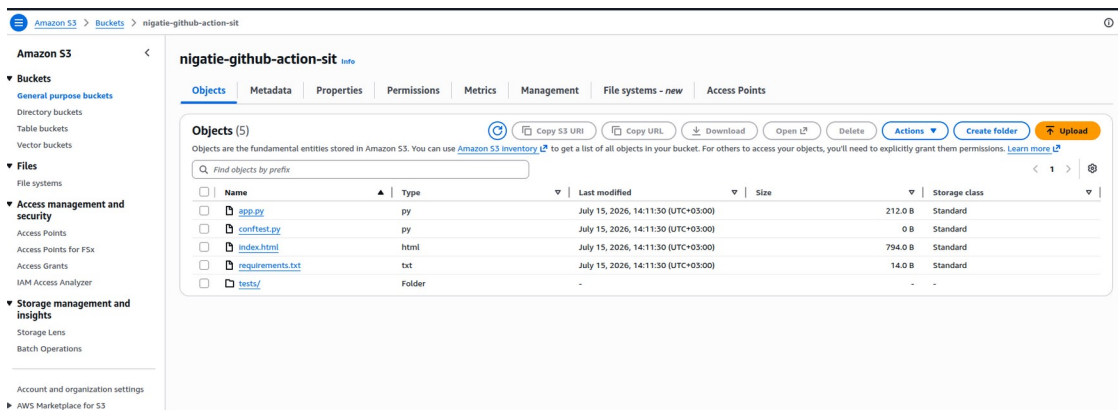


Figure 7 — Bucket contents in the AWS S3 console after sync.



ðŸš€ Deployed from GitHub Actions!

Last deployed: 7/15/2026, 2:15:07 PM

This website was automatically deployed using GitHub Actions CI/CD.

Figure 8 — Live site served from the S3 static website endpoint.

Lab 4: Manual Approval for Production

A production environment was configured under repository Settings > Environments with a required-reviewer protection rule. The deploy job in `deploy.yml` was updated to reference environment: `production`, so any push touching the website files now pauses for manual review before the deployment proceeds. A test commit was pushed to confirm the run paused correctly and completed only after approval.

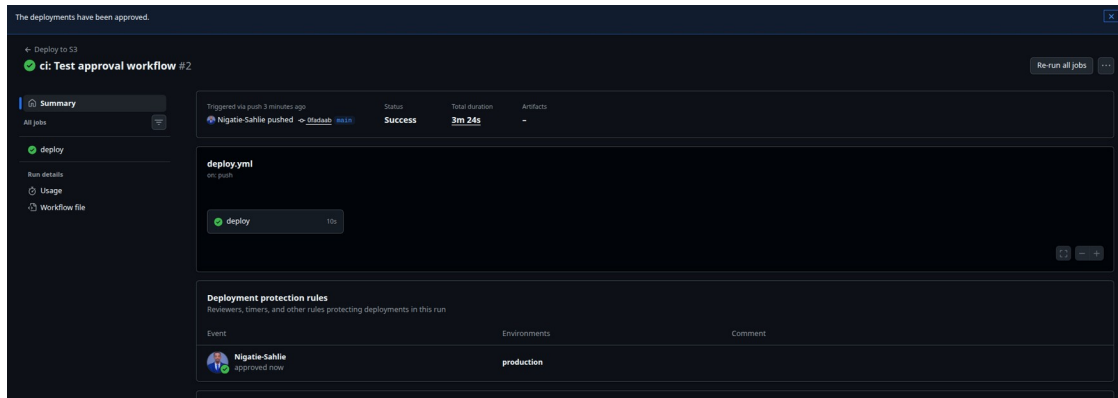


Figure 9 — Deployment approved by reviewer; job completed in 3m 24s including wait time.

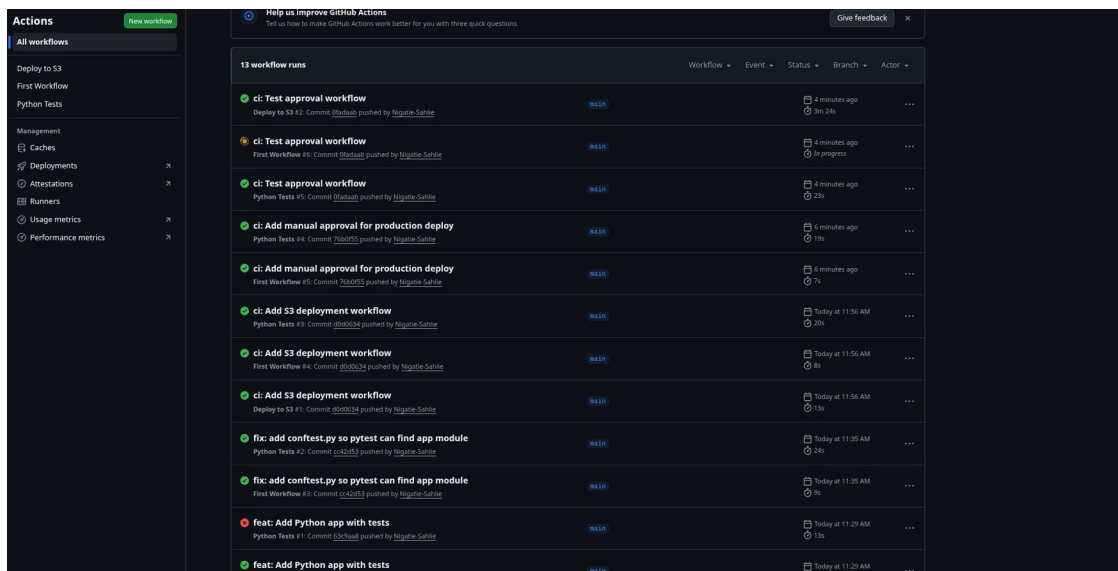


Figure 10 — Full workflow run history showing all four labs completed successfully.

Conclusion

All four labs were completed successfully. Two issues were encountered and resolved independently: a missing `confest.py` causing `pytest` import failures in Lab 2, and a repeated misconfiguration of GitHub repository secrets (entering the key value as the secret name rather than the secret value) in Lab 3, corrected by re-adding the secrets with the correct name/value pairs. The exercise demonstrated a complete CI/CD pipeline: build and test automation, cloud deployment, and a governance control (manual approval) before production release.